

GUIA COMPLETO DE PRODUTIVIDADE

Claude Code

Todos os atalhos, comandos e dicas para
maximizar sua produtividade

Atualizado em Março 2026 | v2.1.72+

Sumario

1. Comandos Slash Essenciais
2. Atalhos de Teclado
3. Flags e Opcoes da CLI
4. Servidores MCP
5. Sistema de Hooks
6. Memoria e Arquivos CLAUDE.md
7. Configuracoes e Settings
8. Integracoes com IDEs
9. Modo de Pensamento Estendido
10. Subagentes e Task Tool
11. Integracoes com Git
12. Gerenciamento de Contexto
13. Otimizacao de Custos
14. Novidades 2025-2026
15. Cheat Sheet Rapido

1. Comandos Slash Essenciais

Os comandos slash sao digitados no prompt interativo do Claude Code. Eles oferecem acesso rapido a todas as funcionalidades.

1.1 Gerenciamento de Sessao

Comando	Descricao
/clear ou /reset ou /new	Limpa o historico da conversa atual e libera contexto
/compact [instrucoes]	Compacta a conversa com instrucoes opcionais de foco
/context	Visualiza o uso de contexto como grade colorida
/copy	Copia a ultima resposta para a area de transferencia
/cost	Mostra estatisticas de uso de tokens
/export [arquivo]	Exporta a conversa como texto para arquivo ou clipboard
/fork [nome]	Cria uma bifurcacao da conversa no ponto atual
/rename [nome]	Renomeia a sessao atual; auto-gera se nenhum nome for dado
/resume [sessao]	Retoma conversa por ID ou nome; abre seletor sem args
/rewind ou /checkpoint	Volta a um ponto anterior ou resume a partir de mensagem
/status	Abre Settings com versao, modelo e info da conta
/stats	Visualiza uso diario, historico de sessoes e streaks
/tasks	Lista e gerencia tarefas em background

1.2 Configuracao e Preferencias

Comando	Descricao
/config ou /settings	Abre a interface de configuracoes
/keybindings	Abre/cria arquivo de configuracao de atalhos
/model [modelo]	Seleciona/troca modelo de IA (sonnet, opus, haiku)
/terminal-setup	Configura atalhos do terminal para Shift+Enter
/theme	Muda tema de cores (light/dark/colorblind/ANSI)
/vim	Alterna entre modo Vim e modo Normal de edicao
/output-style [estilo]	Alterna estilos: Default, Explanatory, Learning
/permissions	Visualiza/atualiza permissoes de ferramentas

1.3 Desenvolvimento e Codigo

Comando	Descricao
/init	Inicializa projeto com arquivo CLAUDE.md guia
/diff	Abre visualizador interativo de diffs
/pr-comments [PR]	Busca comentarios de PR do GitHub (requer gh CLI)

Comando	Descrição
/security-review	Analisa mudancas pendentes para vulnerabilidades
/plan [descricao]	Entra no modo Plan diretamente do prompt
/add-dir <caminho>	Adiciona novo diretorio de trabalho a sessao

1.4 Ferramentas e Integracoes

Comando	Descrição
/agents	Gerencia configuracoes de agentes/subagentes
/chrome	Configura Claude no Chrome
/hooks	Gerencia configuracoes de hooks
/mcp	Gerencia conexoes de servidores MCP e autenticacao
/memory	Edita CLAUDE.md, alterna auto-memory, visualiza entradas
/plugin	Gerencia plugins do Claude Code
/reload-plugins	Recarrega plugins sem reiniciar
/skills	Lista skills disponiveis
/loop <intervalo> <cmd>	Executa prompts/comandos em intervalos recorrentes
/btw <pergunta>	Pergunta rapida lateral sem adicionar ao historico
/doctor	Diagnostica e verifica instalacao e configuracoes

Dica: Use ! no inicio para executar comandos bash diretamente, e @ para referenciar arquivos com autocompletar.

2. Atalhos de Teclado

2.1 Controles Gerais

Atalho	Descrição
Ctrl+C	Cancela entrada atual ou geração em andamento
Ctrl+D	Sai da sessão do Claude Code
Ctrl+L	Limpa a tela do terminal (mantém histórico)
Ctrl+O	Alterna saída detalhada (verbose mode)
Ctrl+R	Busca reversa no histórico de comandos
Ctrl+T	Alterna visibilidade da lista de tarefas
Ctrl+B	Coloca tarefas em background (tmux: pressione 2x)
Ctrl+F	Encerra todos os agentes em background (2x em 3s para confirmar)
Ctrl+G	Abre prompt em editor de texto externo
Ctrl+V (Win: Alt+V)	Cola imagem da área de transferência
Shift+Tab ou Alt+M	Alterna modos de permissão: Auto-Accept / Plan / Normal
Option+P (Mac) ou Alt+P	Troca de modelo durante a sessão
Option+T (Mac) ou Alt+T	Alterna pensamento estendido
Esc + Esc	Volta (rewind) ou resume ponto anterior
Setas Esq/Dir	Navega entre abas em diálogos
Setas Cima/Baixo	Navega pelo histórico de comandos

2.2 Edição de Texto

Atalho	Descrição
Ctrl+K	Deleta até o final da linha
Ctrl+U	Deleta a linha inteira
Ctrl+Y	Cola o texto deletado com Ctrl+K/U
Alt+Y (após Ctrl+Y)	Navega pelo histórico de colagens
Alt+B	Mova cursor uma palavra para trás
Alt+F	Mova cursor uma palavra para frente

2.3 Entrada Multilinha

Método	Atalho	Suporte
Escape rápido	\\ + Enter	Todos os terminais
macOS padrão	Option+Enter	Apenas macOS
Shift+Enter	Shift+Enter	iTerm2, WezTerm, Ghostty, Kitty
Controle	Ctrl+J	Caractere line feed

Metodo	Atalho	Suporte
Colar direto	Cole o texto	Blocos de código, logs

Dica: Execute `/terminal-setup` para configurar `Shift+Enter` no seu terminal. Para usuários avançados, `/vim` habilita o modo de edição Vim completo.

3. Flags e Opcoes da CLI

3.1 Comandos Basicos

Comando	Descricao
claude	Inicia sessao interativa
claude "consulta"	Inicia com prompt inicial
claude -p "consulta"	Modo print: responde e sai (ideal para scripts)
cat arquivo claude -p "consulta"	Processa conteudo via pipe
claude -c	Continua a conversa mais recente no diretorio atual
claude -r "sessao" "consulta"	Retoma sessao especifica por ID ou nome
claude update	Atualiza para a versao mais recente

3.2 Flags de Modelo e Execucao

Flag	Descricao
--model <modelo>	Define modelo: sonnet, opus, haiku ou nome completo
--fallback-model <modelo>	Fallback automatico quando sobrecarregado (modo print)
--max-turns <numero>	Limita turnos agenticos (modo print)
--max-budget-usd <valor>	Para antes de gastar este valor (modo print)
--verbose	Habilita logging detalhado
--debug [categorias]	Modo debug (filtros opcionais: api, hooks)
--output-format <fmt>	text, json ou stream-json (modo print)

3.3 Flags de Permissao

Flag	Descricao
--permission-mode <modo>	Inicia em modo: default, acceptEdits, plan, dontAsk, bypassPermissions
--dangerously-skip-permissions	Pula todos os prompts de permissao (use com cautela!)
--allowedTools <regras>	Padroes de ferramentas pre-aprovadas
--disallowedTools <regras>	Padroes de ferramentas bloqueadas
--tools "Bash,Edit,Read"	Restringe ferramentas disponiveis

3.4 Flags de System Prompt e MCP

Flag	Descricao
--system-prompt <texto>	Substitui o system prompt inteiro
--append-system-prompt <texto>	Adiciona ao system prompt padrao
--mcp-config <caminho>	Carrega servidores MCP de arquivo JSON
--strict-mcp-config	Usa apenas MCP do --mcp-config

Flag	Descricao
--agent <nome>	Especifica subagente para a sessao
--agents '<json>'	Define subagentes customizados via JSON
--worktree, -w [nome]	Inicia em git worktree isolado
--chrome / --no-chrome	Habilita/desabilita integracao com Chrome

4. Servidores MCP

O Model Context Protocol (MCP) permite conectar o Claude Code a ferramentas externas, APIs e fontes de dados. E o principal mecanismo de extensibilidade.

4.1 Instalando Servidores MCP

Tipo	Comando
HTTP (recomendado)	claude mcp add --transport http <nome> <url>
HTTP com auth	claude mcp add --transport http api <url> --header "Authorization: Bearer token"
Stdio (local)	claude mcp add --transport stdio <nome> -- <comando> [args]
Stdio com env	claude mcp add --transport stdio --env KEY=val server -- npx pacote

4.2 Comandos de Gerenciamento

Comando	Descricao
claude mcp list	Lista todos os servidores configurados
claude mcp get <nome>	Detalhes de servidor especifico
claude mcp remove <nome>	Remove servidor
claude mcp add-from-claude-desktop	Importa servidores do Claude Desktop
/mcp (interativo)	Checa status e autentica no modo interativo

4.3 Escopos de Instalacao

Escopo	Flag	Uso
Local	--scope local	Especifico do projeto (~/.claude.json) - padrao
Projeto	--scope project	Compartilhado com o time (.mcp.json)
Usuario	--scope user	Todos os projetos (~/.claude.json)

4.4 Servidores Populares

Servico	Comando de Instalacao
GitHub	claude mcp add --transport http github https://api.githubcopilot.com/mcp/
Sentry	claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
Postgres	claude mcp add --transport stdio db -- npx @bytebase/dbhub --dsn "..."
Notion	claude mcp add --transport http notion https://mcp.notion.com/mcp

4.5 Otimizacao de Ferramentas MCP

Variavel de Ambiente	Descricao
ENABLE_TOOL_SEARCH=auto	Auto-habilita busca a 10% do contexto
ENABLE_TOOL_SEARCH=auto:5	Auto-habilita a 5% do contexto

Variavel de Ambiente	Descricao
ENABLE_TOOL_SEARCH=true	Sempre habilitado
MAX_MCP_OUTPUT_TOKENS=50000	Aumenta limite de saida do MCP

5. Sistema de Hooks

Hooks permitem executar código automaticamente em resposta a eventos do Claude Code, como antes/depois de usar ferramentas, ao iniciar sessões, etc.

5.1 Eventos Disponíveis

Evento	Quando Dispara
SessionStart	Sessão inicia ou é retomada
UserPromptSubmit	Usuário envia prompt (antes do processamento)
PreToolUse	Antes da chamada de ferramenta (pode bloquear)
PostToolUse	Após ferramenta ser executada com sucesso
PostToolUseFailure	Após ferramenta falhar
Notification	Claude envia notificação
SubagentStart / SubagentStop	Subagente inicia / finaliza
Stop	Claude termina resposta
PreCompact	Antes da compactação de contexto
SessionEnd	Sessão termina

5.2 Tipos de Hook

Tipo	Descrição	Exemplo de Uso
command	Executa script shell	Formatar código após edição
http	Faz POST para URL	Notificar webhook externo
prompt	Avaliação LLM single-turn	Validar qualidade de código
agent	LLM multi-turn com ferramentas	Revisão de código complexa

5.3 Códigos de Saída

Código	Efeito
0	Permite a ação (stdout adicionado ao contexto)
2	Bloqueia a ação (stderr vira feedback para o Claude)
Outro	Permite a ação (stderr logado, não mostrado ao Claude)

Dica: Use hooks `PostToolUse` com matcher `'Edit|Write'` para rodar formataadores (Prettier, Black) automaticamente após cada edição de arquivo.

6. Memoria e Arquivos CLAUDE.md

Os arquivos CLAUDE.md sao a 'memoria' persistente do Claude Code. Eles definem instrucoes, preferencias e regras que sao carregadas automaticamente em cada sessao.

6.1 Localizacoes e Prioridade

Escopo	Localizacao	Uso
Politica gerenciada	Diretorios do sistema (/etc/claude-code/)	Regras organizacionais
Projeto	./CLAUDE.md ou ~/.claude/CLAUDE.md	Instrucoes do time
Usuario	~/.claude/CLAUDE.md	Preferencias pessoais
Subdiretorio	src/api/CLAUDE.md	Regras especificas de caminho
Auto-memoria	~/.claude/projects/<projeto>/memory/	Aprendizado automatico

6.2 Boas Praticas

Mantenha o CLAUDE.md abaixo de 200 linhas para melhor aderencia. Use headers e bullets. Seja especifico: 'Use indentacao de 2 espacos' ao inves de 'Formate o codigo corretamente'. Importe arquivos com @caminho/arquivo. Verifique se nao ha regras conflitantes.

6.3 Regras por Caminho (.claude/rules/)

Voce pode criar regras especificas para caminhos de arquivo dentro de .claude/rules/. Use o frontmatter YAML com 'paths:' seguido de padroes glob, e entao escreva as regras abaixo. Exemplo: paths: src/api/**/*.ts aplica regras somente para arquivos TypeScript na pasta api.

6.4 Comandos Uteis

Comando	Descricao
/memory	Edita CLAUDE.md, alterna auto-memory, visualiza entradas
/init	Gera um CLAUDE.md inicial para o projeto

7. Configuracoes e Settings

7.1 Arquivos de Configuracao

Escopo	Localizacao	Precedencia
Gerenciado	Diretorios do sistema	1 (mais alta)
Argumentos CLI	Linha de comando	2
Local	.claude/settings.local.json	3
Projeto	.claude/settings.json	4
Usuario	~/.claude/settings.json	5 (mais baixa)

7.2 Regras de Permissao

Configure permissoes granulares no settings.json usando o campo 'permissions' com arrays 'allow', 'ask' e 'deny'. Exemplos de padroes:

Padrao	Descricao
Bash(npm run *)	Permite comandos que comecam com 'npm run'
Bash(git commit *)	Permite git commit com qualquer argumento
Edit(src/**)	Permite edicao em arquivos dentro de src/
Read(/.env)	Controla acesso ao arquivo .env
WebFetch(domain:github.com)	Controla acesso ao dominio

7.3 Sandbox (Isolamento)

O sandbox oferece isolamento seguro com controle granular de filesystem e rede. Configure em settings.json com: `sandbox.enabled` (true/false), `sandbox.filesystem.allowWrite` e `sandbox.filesystem.denyRead` para controlar acesso a arquivos, e `sandbox.network.allowedDomains` para controle de rede.

7.4 Configuracoes de Modelo

Use 'model' para definir o modelo padrao, 'availableModels' para restringir opcoes, 'outputStyle' para estilo de saida, e 'alwaysThinkingEnabled' para ativar pensamento estendido permanentemente.

8. Integracoes com IDEs

8.1 VS Code

Recurso	Descricao
Instalacao	code --install-extension Anthropic.claude-code
Configuracao	/keybindings para personalizar atalhos
Recursos	Retomar conversas, multi-sessao, integracao Git
Terminal	/terminal-setup para configurar atalhos do terminal
MCP (v2.1.70+)	Dialog de gerenciamento MCP integrado ao VS Code

8.2 JetBrains (IntelliJ, PyCharm, WebStorm...)

Recurso	Descricao
Instalacao	Marketplace da IDE
Configuracao	Plugin Settings nas preferencias da IDE
Tecla Escape	Configure "ESC Key Configuration" no plugin
Remoto	Suporte a WSL e SSH
Inicializacao	claude --ide para auto-conectar

9. Modo de Pensamento Estendido

O pensamento estendido permite que o Claude 'pense' mais profundamente antes de responder, ideal para problemas complexos de raciocinio.

9.1 Controles de Pensamento

Metodo	Descricao
Option+T (Mac) / Alt+T	Alterna pensamento estendido na sessao
/config	Define alwaysThinkingEnabled como padrao
CLAUDE_CODE_EFFORT_LEVEL=high	Define budget de pensamento alto
MAX_THINKING_TOKENS=10000	Limita tokens de pensamento
CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING=1	Usa budget fixo
Ctrl+O	Visualiza pensamento no modo verbose
"ultrathink" no prompt	Inclua no prompt para esforco alto

9.2 Niveis de Esforco (v2.1.68+)

Nivel	Simbolo	Uso
Baixo	■	Tarefas simples, respostas rapidas
Medio	■	Padrao para Opus 4.6, equilibrio
Alto	●	Problemas complexos, raciocinio profundo

Dica: Use niveis de esforco baixo para tarefas simples e economize tokens. Reserve 'ultrathink' para problemas realmente complexos de arquitetura ou debugging.

10. Subagentes e Task Tool

Subagentes permitem delegar tarefas específicas a agentes especializados, cada um com suas próprias ferramentas, modelo e permissões.

10.1 Subagentes Nativos

Agente	Modelo	Ferramentas	Uso
Explore	Haiku	Somente leitura	Busca de arquivos, exploração do codebase
Plan	Herda	Somente leitura	Planejamento antes de implementação
General-purpose	Herda	Todas	Operações complexas multi-passo

10.2 Agentes Customizados

Crie agentes customizados com `/agents` ou definindo arquivos Markdown em `.claude/agents/` (projeto) ou `~/.claude/agents/` (usuário). Cada agente pode ter: ferramentas específicas, modelo dedicado, modo de permissão, limite de turnos, skills associadas e isolamento em `worktree`.

10.3 Escopos de Agentes

Escopo	Localização
Sessão	Flag <code>--agents</code> na CLI
Projeto	<code>.claude/agents/</code>
Usuário	<code>~/.claude/agents/</code>
Plugin	Diretório <code>agents/</code> do plugin

10.4 Memória Persistente de Agentes

Agentes podem ter memória persistente configurando `'memory: user'` (em `~/.claude/agent-memory/`) ou `'memory: project'` para memória por projeto.

11. Integrações com Git

11.1 Comandos Git Integrados

Comando	Descrição
claude --worktree feature-auth	Cria e inicia em worktree nomeado
claude --worktree	Auto-gera nome para o worktree
/diff	Visualizador interativo de diffs (git diff + diffs por turno)
/pr-comments [#123]	Busca comentários de PR do GitHub
claude --from-pr 123	Retoma sessão vinculada a PR
/security-review	Analisa git diff para vulnerabilidades de segurança

11.2 Worktrees

Worktrees oferecem cópias isoladas do repositório sem colisões. São automaticamente limpos se nenhuma mudança for feita. Subagentes podem usar 'isolation: worktree' na configuração para trabalhar em worktrees separados.

Dica: Use worktrees para experimentar sem afetar a branch principal. Combine com subagentes para ter múltiplas linhas de trabalho paralelas.

12. Gerenciamento de Contexto

Gerenciar o contexto eficientemente e crucial para sessoes produtivas e economicas.

12.1 Monitoramento

Comando	Descricao
/context	Visualiza uso de contexto como grade colorida
/cost	Mostra tokens usados e custo (usuarios API)
/stats	Padroes de uso diario e preferencias
/usage	Limites do plano e status de rate limit
/statusline	Configura barra de status com info em tempo real

12.2 Estrategias de Otimizacao

Estrategia	Como Implementar
Prompts especificos	"Corrija bug em auth.ts" ao inves de "Melhore o codigo"
Limpar entre tarefas	/rename a sessao atual, depois /clear
Modelo adequado	Sonnet para maioria, Opus para raciocinio complexo
Reduzir overhead MCP	/context para identificar servidores pesados
Hooks para pre-processamento	Filtrar logs antes do Claude ler
Instrucoes em skills	Mantenha CLAUDE.md abaixo de 500 linhas
Delegar a subagentes	Trabalho verbose fora do contexto principal
Trabalhar incrementalmente	Teste cedo, pegue problemas barato
Compactar com foco	/compact "foco nas mudancas de API"

13. Otimizacao de Custos

13.1 Estrategias de Reducao de Custo

Estrategia	Impacto	Implementacao
Escolha de modelo	Alto	Sonnet < Opus; Haiku para subagentes
Gerenciamento de contexto	Alto	/context, /compact, /clear entre tarefas
Controle de pensamento	Medio	Baixe MAX_THINKING_TOKENS para tarefas simples
Planejar antes de implementar	Medio	Use Plan Mode para evitar retrabalho
Tool search automatico	Medio	ENABLE_TOOL_SEARCH=auto:5
Subagentes	Medio	Delegue tarefas verbose a subagentes
Hooks eficientes	Baixo	Pre-processe e filtre antes do Claude

13.2 Limites de Rate por Tamanho de Time

Tamanho do Time	TPM/Usuario	RPM/Usuario
1-5 pessoas	200k-300k	5-7
5-20 pessoas	100k-150k	2.5-3.5
20-50 pessoas	50k-75k	1.25-1.75
50-100 pessoas	25k-35k	0.62-0.87
100-500 pessoas	15k-20k	0.37-0.47
500+ pessoas	10k-15k	0.25-0.35

14. Novidades 2025-2026

14.1 Ultimas Versoes

Versao	Recurso	Impacto
v2.1.72	Loop/Cron: /loop 5m <comando>	Prompts recorrentes automaticos
v2.1.72	Voice STT em 20 idiomas	Entrada por voz expandida
v2.1.72	Exibicao de nivel de esforco	Transparencia no processamento
v2.1.71	Push-to-talk rebindavel	Controle de voz personalizado
v2.1.70	MCP Dialog no VS Code	Gerenciamento MCP visual
v2.1.70	Plan Document View	Planos com comentarios
v2.1.69	Claude API Skill (/claude-api)	Acesso a docs da API
v2.1.69	/reload-plugins	Plugins sem reiniciar
v2.1.69	Skills hooks (InstructionsLoaded)	Hooks em skills
v2.1.68	Opus 4.6 como padrao (esforco medio)	Modelo mais poderoso
v2.1.68	Niveis de esforco simplificados	low/medium/high (3 niveis)

15. Cheat Sheet Rapido

15.1 Fluxo de Trabalho Diario

Acao	Comando
Iniciar sessao	claude
Nomear trabalho	/rename nome-da-tarefa
Retomar trabalho	/resume nome-da-tarefa
Verificar contexto	/context
Ver custos	/cost
Trocar modelo	/model (ou Alt+P)
Ajustar permissoes	Shift+Tab
Revisar mudancas	/diff
Comprimir historico	/compact
Modo verbose	Ctrl+O
Limpar tela	Ctrl+L
Interromper	Ctrl+C
Voltar (rewind)	Esc + Esc

15.2 Fluxo de Sessao Ideal

Passo	Comando	Objetivo
1	claude --permission-mode plan	Exploracao segura
2	/plan detalhes da implementacao	Criar plano
3	Shift+Tab (Auto-Accept)	Modo de edicao
4	(faca as mudancas)	Implementacao
5	/diff	Revisar mudancas
6	Shift+Tab (Plan)	Voltar ao modo seguro
7	/compact	Limpar contexto
8	/rename	Salvar sessao

15.3 Atalhos Mais Importantes

Atalho	Para Que Serve
Shift+Tab	Alterna modos de permissao (o mais util!)
Alt+P ou Option+P	Troca modelo rapidamente
Alt+T ou Option+T	Liga/desliga pensamento estendido
Ctrl+O	Modo verbose (ve o que Claude pensa)
Esc + Esc	Desfaz/volta no historico

Atalho	Para Que Serve
Ctrl+G	Editor externo para prompts longos
@arquivo	Referencia arquivos com autocompletar
!comando	Executa bash diretamente

Guia gerado em Marco de 2026 com base na documentacao oficial do Claude Code v2.1.72+. Informacoes sujeitas a atualizacao.